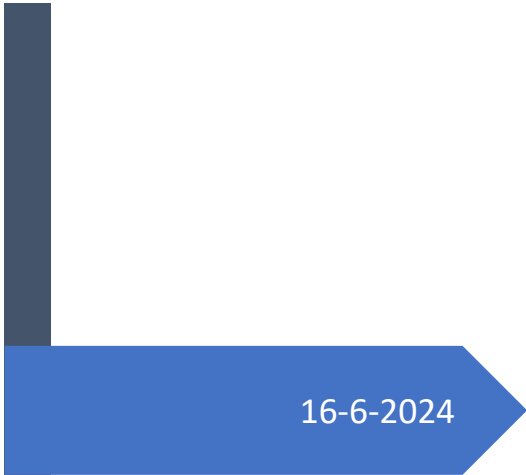
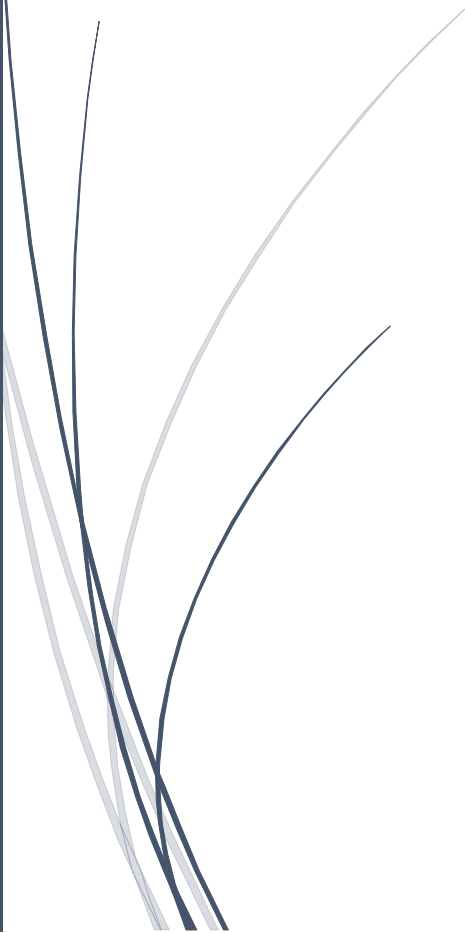


A dark blue vertical bar on the left side of the slide. A blue arrow points to the right from the bar, containing the date.

16-6-2024

Deadlocks en Sistemas Operativos

Enfoques utilizados en Windows
y GNU/Linux

Several thin, curved lines in dark blue and light grey originate from the bottom left and curve upwards and to the right.

Antonio Felix Glorioso Ceretti
Agustina Sol Rojas
SISTEMAS OPERATIVOS

Introducción

En la actualidad los Sistemas Operativos permiten la ejecución simultanea de varias tareas a través de una herramienta denominada hilos. Estos subprocesos requieren el uso de ciertos recursos (CPU, memoria, etc.) para completar sus determinadas tareas. Es aquí donde encontramos ciertas situaciones en las que dos o más procesos del mismo conjunto esperan a que cada uno de ellos libere los recursos que el otro necesita, bloqueándose entre sí. Esto ultimo se conoce como deadlock.

Existen varios enfoques sobre cómo tratar estas situaciones y cada uno implementa una serie de estrategias para resolver este problema. En este informe explicaremos cuales son, como se utilizan en diferentes Sistemas Operativos y que otras técnicas se podrían implementar.

Deadlocks

Los deadlocks anteriormente mencionados solo ocurren cuando se cumplen en simultaneo estas cuatro condiciones:

1. Exclusión mutua: en un instante de tiempo dado, solo un proceso puede utilizar una instancia de un recurso.
2. Retención y espera: los procesos mantienen los recursos asignados mientras esperan por los nuevos.
3. No apropiación: a un proceso no se le puede sacar forzosamente los recursos que ya tiene.
4. Espera circular: el proceso forma parte de una lista circular en la que cada uno de ellos está esperando por al menos un recurso asignado a otro proceso de la lista.

Enfoques para su tratamiento

Para poder resolver estas situaciones y que el flujo del programa siga con su ejecución normal, nos encontramos con las siguientes estrategias:

Ignorar

Esta estrategia se sostiene bajo la idea de la baja probabilidad de que ocurra un deadlock. Se ignora el problema y se espera que nunca se produzca un interbloqueo en el sistema.

Lo bueno de este enfoque es que se evitan las complejidades y bajas de rendimiento al implementar cualquiera de las otras técnicas. Esto tiene su contraparte, dado que al ocurrir esta situación de interbloqueo será necesario desbloquear el sistema o eliminar manualmente un conjunto de procesos constantemente.

Prevenir / Evitar

Prevenir

Se basa en tratar de asegurar que no se cumple alguna de las 4 condiciones anteriormente mencionadas. Aplica restricciones en la forma en que los procesos requieren los recursos. De acuerdo a la situación y estado del sistema se elige alguna de las siguientes condiciones para prevenir:

1. Exclusión mutua

Se asigna una abstracción del recurso, se simula que este es compartible. Por ejemplo: se pueden implementar listas manejadas por un proceso global que organice el acceso a los recursos. Igualmente, esta estructura podría llenarse y bloquear a los procesos. Existen situaciones donde los recursos no pueden ser simulados como compartibles.

2. Retención y espera

Si un proceso requiere un recurso que no está disponible, que libere aquellos que posee. Esto no siempre será posible y además puede impedir que el proceso avance en su programación.

Existen ciertas alternativas (que siguen siendo ineficientes) para implementar:

- Un proceso debe requerir y reservar todos los recursos a usar antes de comenzar la ejecución.
- El proceso puede requerir recursos sólo cuando no tiene ninguno.

3. No apropiación

Existen diferentes técnicas para esta condición en específico, pero de ellas la menos traumática es:

1. Virtualizar recursos: El proceso no accede directamente al recurso, sino que ingresa a un demonio que lo administra. A medida que los procesos requieren el recurso, interactúan con este demonio quien almacena los trabajos en una cola. Se logra que el recurso físico pueda ser “utilizado” por varios procesos al mismo tiempo (una forma virtual de compartir el recurso).

4. Condición de Espera circular

Se define un ordenamiento de los recursos de la siguiente forma. La función f asigna en orden creciente un numero natural único a cada recurso.

$$f(R_i) = x$$

Donde R_i es un recurso cualquiera y x un numero natural asignado. Si un proceso, que ya tiene un R_i requiere un R_j solo se lo dará si y solo si $f(R_j) > f(R_i)$. No es eficiente realizar esto.

Evitar

Se basa en tomar decisiones de asignación en cuanto al estado del sistema. Se asignan cuidadosamente los recursos manteniendo información actualizada sobre el requerimiento y uso de los mismos. Esto baja la performance del sistema y puede producir baja utilización de recursos.

Se utiliza un algoritmo de evitación de interbloqueo que examina dinámicamente el estado de asignación de recursos para evaluar las posibles consecuencias ante cada solicitud. De acuerdo a lo que analice este programa se puede estar en un estado seguro si existe una “cadena segura” de procesos que puedan ejecutarse con todos los recursos disponibles sin que haya deadlock. Si no existe cadena segura se está en estado inseguro.

Algoritmos para evitar el deadlock

Si se tiene una sola instancia por recurso: se utiliza un algoritmo que determina el estado seguro de un sistema aprovechando el grafo de asignación de recursos. Una secuencia segura es evitar los bucles.

Si se tienen varias instancias por recurso: se utiliza un algoritmo del banquero, que busca la cadena segura simulando las posibles asignaciones de recursos a los procesos.

Detección y Recuperación

Es una técnica que permite determinar si hay un deadlock y si lo hubo poder recuperarse del mismo. Existen dos tipos de algoritmos:

1. Algoritmo que examine si ocurrió un deadlock.
2. Algoritmo para recuperación del deadlock.

Enfoques de las estrategias en diferentes SO

Linux

Por defecto Linux utiliza la estrategia de “Ignorar” ya que, como dice *Fundamentos de Sistemas Operativos (Silberschatz et al., 2006b, pág. 223)* sobre este tópico:

“...es problema del desarrollador de aplicaciones el escribir programas que resuelvan posibles interbloqueos.”

Aun así, se puede activar en la configuración del kernel la herramienta de debugging *lockdep* que permite a Linux añadir wrappers sobre las llamadas de bloqueo al kernel. Cada vez que se toma o libera un tipo específico de bloqueo, se registra este hecho, junto con detalles auxiliares como, por ejemplo, si el procesador estaba atendiendo una interrupción en ese momento. Lockdep también registra qué otros bloqueos ya se tenían cuando se toma el nuevo bloqueo.

Esta información se la devolverá al usuario y este deberá solucionar la situación de acuerdo a donde se encuentre el problema.

Lockdep es capaz de detectar escenarios de bloqueo mutuo más complicados que los descritos anteriormente, como bloqueos que no son seguros para interrupciones que se adquieren en un contexto de interrupción.

Ejecutar un kernel con Lockdep activado tiende a ralentizar las cosas considerablemente; no es una opción que se suele habilitar en un sistema de producción.

En resumen, cuando está habilitado, crea un modelo detallado de cómo se usan los bloqueos en el kernel. Este modelo se puede utilizar para encontrar posibles deadlocks y otros problemas.

Para activarlo se debe editar el archivo `.config` ejecutando:

```
make menuconfig
```

y habilitar en Hacking Options:

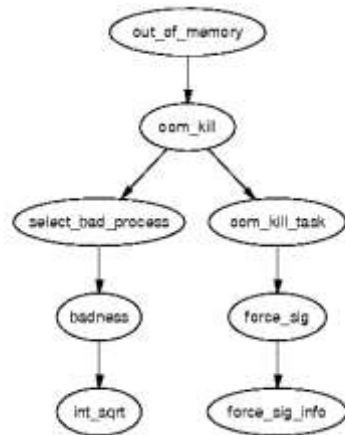
1. [*] Detect Hard and Soft Lockups
2. [*] Detect Hung Tasks
3. [*] RT Mutex debugging, deadlock detection
4. *- Spinlock and rw-lock debugging: basic checks
5. *- Mutex debugging: basic checks
6. *- Lock debugging: detect incorrect freeing of live locks
7. [*] Lock debugging: prove locking correctness
8. [*] Lock usage statistics

Luego se deberá recompilar el kernel, instalarlo y reiniciar el PC.

Hay que tener en cuenta que existen casos de deadlocks que pueden ocasionar la utilización excesiva de la memoria RAM. Esto haría que el SO Linux utilice el Out Of Memory Management (OOM) killer para terminar el proceso, y de esta forma liberar el recurso. Como se ve en el gráfico de la siguiente página, el OOM primero chequea si el sistema no tiene memoria a través de la función `out_of_memory()` y dependiendo su resultado llama a `oom_kill()`. Este primero elegirá que proceso terminar a través del siguiente calculo:

```
badness_for_task = total_vm_for_task / (sqrt(cpu_time_in_seconds) *  
sqrt(sqrt(cpu_time_in_minutes)))
```

Y luego llamara a `oom_kill_task()` para hacer el trabajo.



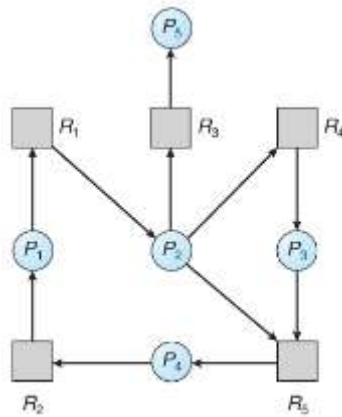
Si bien esta es una forma de resolver el problema, no combate el deadlock ni fue diseñado para esta tarea, solo es funcional en estos casos específicos.

Windows

Windows, como Linux, por defecto también utiliza la estrategia de “Ignorar”. Aun así, este SO tiene una herramienta activable para tratar de combatir esta problemática. Esta se llama “Detección de interbloqueos” y se utiliza solo para los Drivers. La documentación oficial de Windows, en su Artículo de *Detección de interbloqueos* (Domars, 2023), la describe como:

“La detección de interbloqueos supervisa el uso del controlador de los recursos que deben bloquearse: bloqueos de número, exclusiones mutuas y exclusiones mutuas rápidas. Esta opción comprobador de controladores detectará la lógica de código que tiene la posibilidad de provocar un interbloqueo en algún momento futuro.”

Esta herramienta, para buscar posibles deadlocks, utiliza el comprobador de controladores que crea un grafo del orden de adquisición de recursos y comprueba si hay bucles. Este es similar al visto en clase, donde un nodo representa un recurso (cuadrado), otro nodo representa el proceso (circulo) y las flechas son sus pedidos de adquisición. Los bucles representarían infracciones de jerarquía de bloqueo.



El comprobador de controladores emitirá una comprobación de errores cuando se detecte una de estas infracciones. Esto ocurrirá antes de que se produzcan interbloqueos reales. Es en este paso que se puede utilizar la extensión !deadlock que muestra información sobre los interbloqueos recogidos por la opción de detección de interbloqueos del comprobador de controladores. Esta extensión muestra la topología de jerarquía de bloqueo, así como las pilas de llamadas para cada subproceso en el momento en que se adquirieron originalmente los bloqueos.

!deadlock solo proporcionará información útil si la opción de detección de interbloqueo del comprobador de controladores ha detectado una infracción de la jerarquía de bloqueo y ha emitido la comprobación de errores 0xC4 (DRIVER_VERIFIER_DETECTED_VIOLATION).

Si no se utiliza esta extensión el comprobador de controladores emitirá la comprobación de errores 0xC4. El primer parámetro de esta comprobación de errores indicará la infracción exacta.

Para activar esta funcionalidad se deberá utilizar el “Administrador de comprobadores de controladores” o la línea de comandos “Verifier.exe”.

- En la línea de comandos:

Para activar la detección de interbloqueos, usar un valor de marca de 0x20 o agregar 0x20 al valor de marca. Por ejemplo:

```
verifier /flags 0x20 /driver MyDriver.sys
```

La característica estará activa después del siguiente arranque.

La característica “Detección de interbloqueos” también se incluye en la configuración estándar. Por ejemplo:

```
verifier /standard /driver MyDriver.sys
```

- Uso del “Administrador de comprobadores de controladores”:
 1. Seleccionar “Crear configuración personalizada (para desarrolladores de código)” y, a continuación, seleccionar “Siguiente”.
 2. Seleccionar “Seleccionar configuración individual en una lista completa”.
 3. Seleccionar “(comprobar) Detección de interbloqueos”.

Validez y estrategias propuestas por ChatGPT

Respuesta de ChatGPT

“Validación del Informe

El informe es claro y detalla bien los conceptos y métodos actuales. A continuación, propongo otros enfoques que podrían implementarse en ambos sistemas operativos para manejar los deadlocks.

Propuestas de Nuevos Enfoques

Linux

Enfoque Propuesto: Uso de Algoritmos de Evitación de Deadlocks. En lugar de solo depender de herramientas como Lockdep, se podría implementar un sistema que utilice algoritmos de evitación de deadlocks como el algoritmo del banquero. Este enfoque permite al sistema operativo evaluar continuamente el estado de los recursos y las solicitudes de procesos para asegurar que el sistema nunca entre en un estado inseguro.

Implementación:

1. *Monitoreo Continuo: Integrar un módulo de monitoreo que rastree todas las solicitudes de recursos y el estado actual de los recursos.*
2. *Algoritmo del Banquero: Implementar el algoritmo del banquero en el kernel de Linux para verificar cada solicitud de recurso en tiempo real. El algoritmo asegurará que no se concedan recursos si esto puede llevar a un estado inseguro.*
3. *Adaptabilidad Dinámica: Permitir al sistema ajustar dinámicamente los recursos en función de las necesidades de los procesos, utilizando técnicas de predicción para prever demandas futuras.*

Windows

Enfoque Propuesto: Implementación de Protocolos de Preasignación Windows podría beneficiarse de un sistema de preasignación de recursos donde los procesos deben declarar anticipadamente sus necesidades de recursos. Esto puede ayudar a gestionar y prevenir situaciones de deadlock.

Implementación:

- 1. Declaración de Recursos: Modificar el manejador de procesos para requerir que cada proceso declare todos los recursos que necesitará antes de comenzar la ejecución.*
- 2. Asignación Basada en Prioridad: Introducir un sistema de prioridad que asigne recursos basándose en la importancia y urgencia de los procesos.*
- 3. Simulación y Validación: Antes de la ejecución, simular el uso de recursos declarados para asegurar que no se produzca un deadlock. Esto se puede realizar utilizando técnicas de análisis de grafos similares a las utilizadas en la herramienta de detección de interbloqueos de controladores.”*

Análisis de la respuesta de ChatGPT

ChatGPT, en el caso de Linux, está proponiendo el uso del enfoque “Evitar” a través del algoritmo del banquero y técnicas de predicción sobre demandas. Con respecto a Windows este propone usar la estrategia “Prevenir” aplicándola a una de las alternativas de la condición de “Retención y Espera”, agregándole un análisis sobre grafos que simula el uso de recursos.

Creemos que son técnicas adecuadas que se podrían implementar en estos Sistemas Operativos, pero llevarían a pérdidas de la performance sobre el sistema y abuso de la CPU por el propio kernel. Estas serían más provechosas si el sistema tiende a generar deadlocks con frecuencia.

Nosotros proponemos, como otro enfoque, el uso de *machine learning* para tener en cuenta todos los casos. Este aprendería sobre las problemáticas del sistema y su tendencia a deadlocks. De acuerdo a esto podría determinar el enfoque adecuado a utilizar de forma dinámica (en base al actual estado del sistema). Este enfoque podría ser

el uso de Estrategias como “Prevenir y Evitar” o solo dejar que ocurra el deadlock, en los casos donde este sea poco frecuente.

Bibliografía

- Domars. (2023b, junio 15). *Detección de interbloqueos - Windows drivers*. Microsoft Learn. <https://learn.microsoft.com/es-es/windows-hardware/drivers/devtest/deadlock-detection>
- Domars. (2024, 18 abril). *!deadlock (WinDbg) - Windows drivers*. Microsoft Learn. <https://learn.microsoft.com/es-es/windows-hardware/drivers/debuggercmds/-deadlock>
- Facultad de Ingeniería, UDELAR. (2024). *Deadlocks* [Diapositivas; PowerPoint]. https://eva.fing.edu.uy/pluginfile.php/330949/mod_resource/content/4/clase15.pdf
- *How to use lockdep feature in linux kernel for deadlock detection*. (s. f.). Stack Overflow. <https://stackoverflow.com/questions/20892822/how-to-use-lockdep-feature-in-linux-kernel-for-deadlock-detection>
- *Interrupts, threads, and lockdep* [LWN.net]. (s. f.). <https://lwn.net/Articles/321663/>
- Miranda, M. (s. f.). *Linux Kernel Documentation :: LockDep-design.txt*. Copyright (C) 2014 Mauriat Miranda. <https://mjm wired.net/kernel/Documentation/lockdep-design.txt>
- *Out of memory management*. (s. f.). <https://www.kernel.org/doc/gorman/html/understand/understand016.html>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2006c). *Fundamentos de sistemas operativos*.
- *User-space lockdep* [LWN.net]. (s. f.-b). <https://lwn.net/Articles/536363/>